

بخش اول

مقدمه

در این بخش به دنبال این هستیم که جداسازی منابع کور را با فرض استقلال منابع حل کنیم.

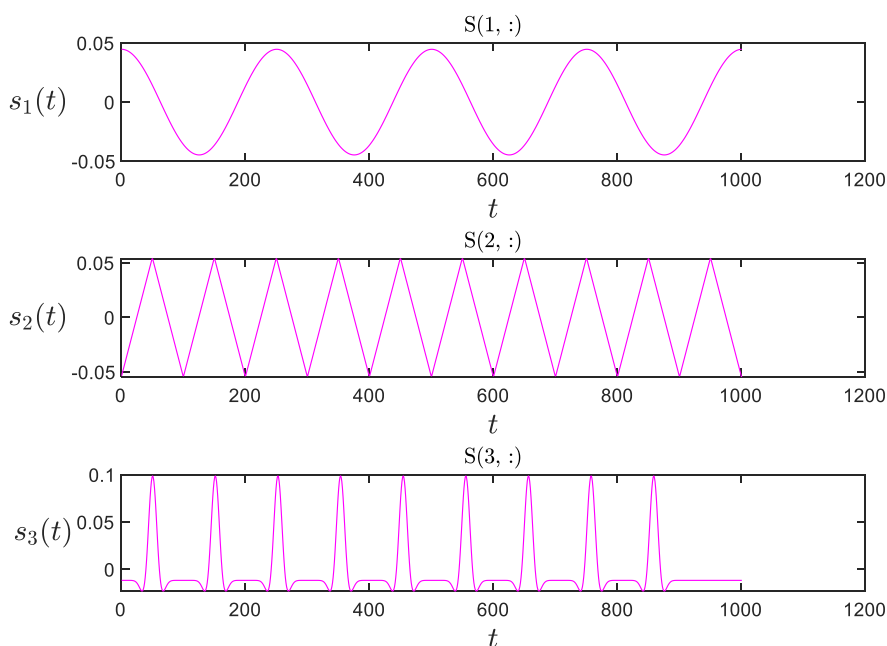
در قسمت ۰، ابتدا منابع و مشاهدات بدون نویز و مشاهدات نویزی را رسم می‌کنیم.

سپس در قسمت ۱، به دنبال این هستیم که ماتریس جدا کننده را بدست آوریم. در ادامه و در قسمت ۲، ابهام منابع و ابهام ترتیب را حذف می‌کنیم و خطا را بدست می‌آوریم.

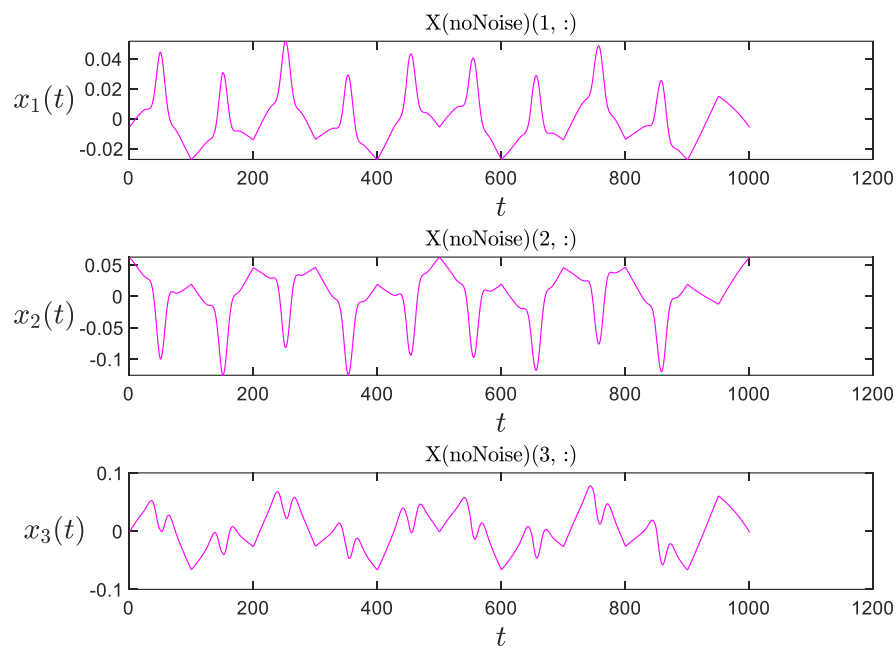
در قسمت ۳، نمودار همگرایی را رسم و نتایج بدست آمده را گزارش می‌کنیم.

قسمت ۰

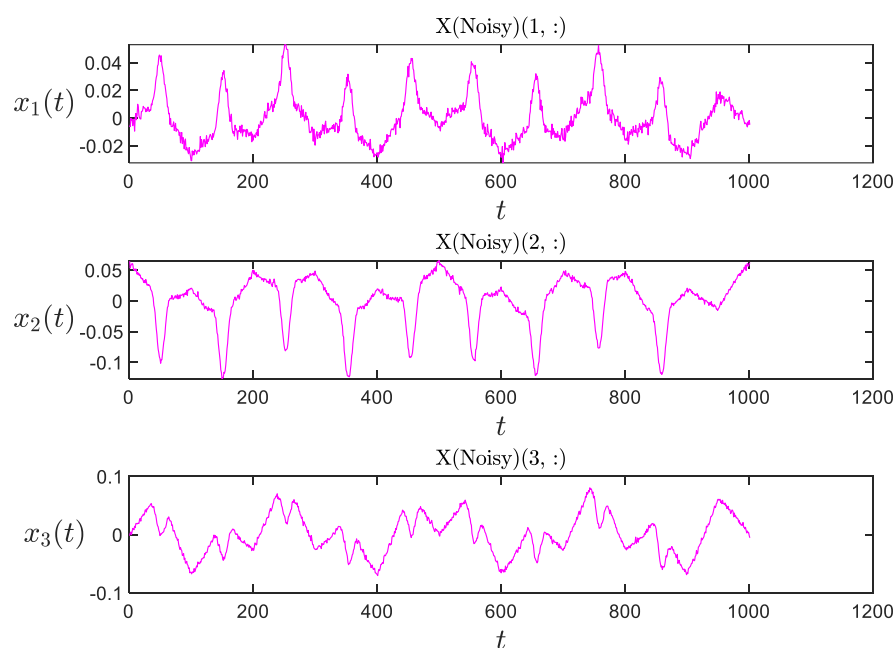
ابتدا از فرمول های $X = A S + Noise$ ، ماتریس مشاهدات نویزی را بدست می‌آوریم. همچنین، از فرمول $X = A S$ نیز مشاهدات بدون نویز بدست می‌آید. حال، نمودارهای خواسته شده را رسم می‌کنیم. تصویر ۱.۱ نمودار منابع، تصویر ۱.۲ نمودار مشاهدات بدون نویز و تصویر ۱.۳ نمودار مشاهدات نویزی را نشان می‌دهد.



تصویر ۱.۱: منابع



تصویر ۱.۲: مشاهدات بدون نویز



تصویر ۱.۳: مشاهدات نویزی

قسمت ۱

در این قسمت به دنبال پیاده سازی روش بیشینه سازی $kurt$ با رویکرد $deflation$ هستیم. به طور خلاصه، این روش به صورت زیر می باشد:

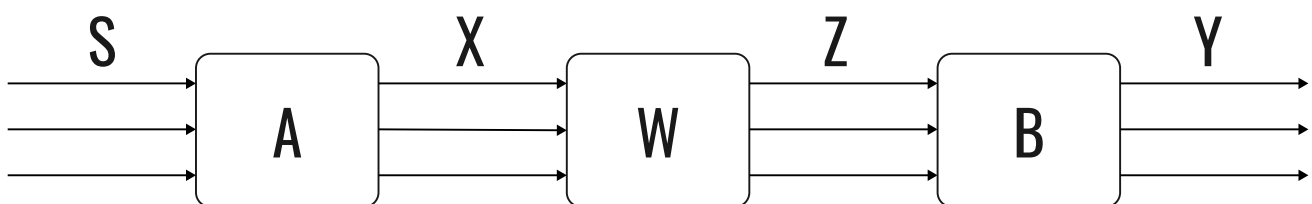
$$f(B) = \sum_{m=1}^M H(y_m) \rightarrow \begin{cases} b_i^{(k+1)} \leftarrow b_i^{(k)} + \mu \frac{\partial f}{\partial b_i} \\ b \leftarrow \frac{b}{\|b\|_2} \\ b_i = \left(I - \begin{bmatrix} b_1^T \\ b_2^T \\ \vdots \\ b_{i-1}^T \end{bmatrix} \right) \end{cases}$$

Score Function: $|kurt(y)|$

$$kurt(y) = E\{y^4\} - 3(E\{y^2\})^2 \rightarrow \text{if } y \sim N(0, \sigma^2) \Rightarrow kurt(y) = 0$$

$$\rightarrow \frac{\partial f}{\partial b} = \text{sign}(kurt(b^T Z)) [E\{Z(b^T Z)^3\} - 3b]$$

تصویر ۱.۴، نمودار کلی مسئله BSS در این الگوریتم را نشان می دهد.



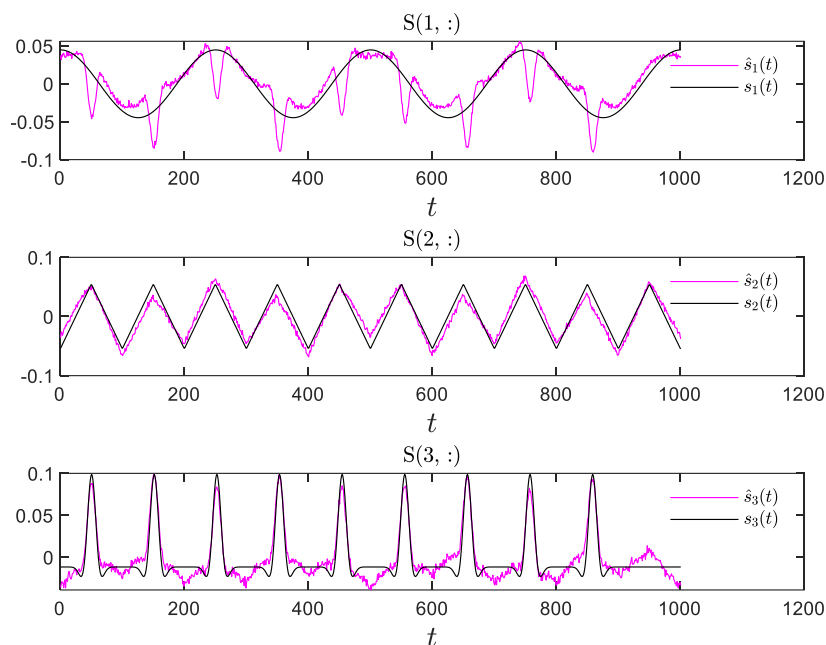
تصویر ۱.۴: نمودار کلی مسئله BSS برای deflation kurt

حال، ماتریس بدست آمده از ضرب ماتریس جدا کننده و ماتریس مخلوط کننده اصلی را بدست می آوریم. این ماتریس از رابطه $\text{permutation_matrix} = B \times W \times A$ بدست می آید. این ماتریس به صورت زیر می باشد:

$$\text{permutation_matrix} = \begin{bmatrix} -0.9897 & -0.0699 & 0.0751 \\ -0.0421 & 1.0721 & -0.9009 \\ 0.0179 & 0.4248 & 0.6969 \end{bmatrix}$$

همان طور که مشاهده می شود درایه نشان داده شده باعث شده تا ماتریس بدست آمده از permutation matrix دور باشد.

تصویر ۱.۵ مقایسه منابع تخمین زده شده از مشاهدات نویزی با منابع اصلی را نشان می دهد.



تصویر ۱.۵: مقایسه منابع تخمین زده شده از مشاهدات نویزی با منابع اصلی

قسمت ۲

رابطه خطا از فرمول زیر بدست می آید:

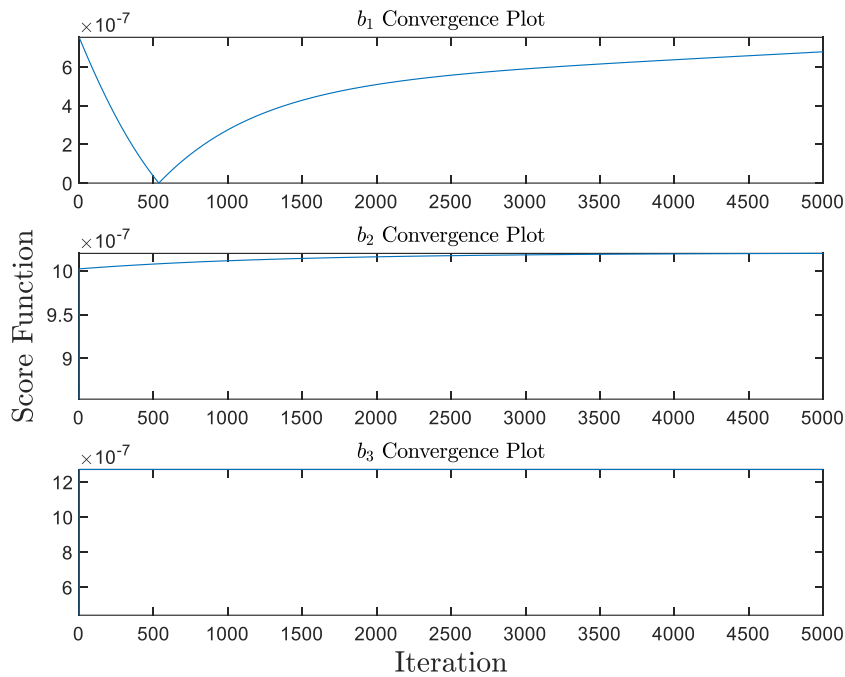
$$E = \frac{\|\hat{S} - S\|_F^2}{\|S\|_F^2}$$

نتایج بدست آمده از یک بار اجرای کد به صورت زیر می باشد:

$$E = 0.21409$$

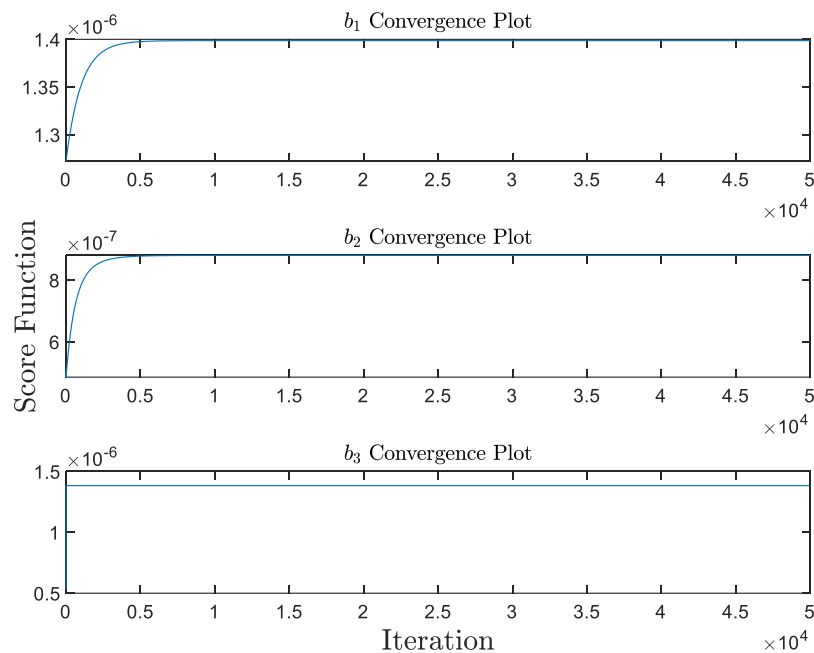
قسمت ۳

تابع هدف برابر با $f(B) = |kurt(y)|$ می باشد. نمودار آن را بر حسب شماره **iteration** رسم می کنیم. نمودار این تابع هدف در تصویر ۱.۶ آمده است.



تصویر ۱.۶: نمودار تابع هدف بر حسب شماره iteration (5000 iteration)

برای مشاهده بهتر این تابع هدف، تعداد iteration ها را بیشتر می کنیم (10 برابر می کنیم) و مجدداً این نمودار را رسم می کنیم. تصویر ۱.۷ این نمودار را نشان می دهد.



تصویر ۱.۷: نمودار تابع هدف بر حسب شماره iteration (50000 iteration)

همان طور که مشاهده می شود، تابع هدف مورد نظر طی iteration ها ماکزیمم می شود و مقدار آن از یک iteration به بعد، ثابت می شود.

در نمودار پایینی تصویر ۱.۷، مشاهده می‌شود که به علت یکتا بودن سطر سوم به علت تعامد، این سطر در یک iteration یافت شده است.

بخش دوم

مقدمه

در این بخش به دنبال این هستیم که جداسازی منابع کور را با فرض استقلال منابع حل کنیم.

در قسمت ۱، به دنبال این هستیم که ماتریس جدا کننده را بدست آوریم. در ادامه و در قسمت ۲، ابهام منابع و ابهام ترتیب را حذف می کنیم و خطا را بدست می آوریم.

در قسمت ۳، نمودار همگرایی را رسم و نتایج بدست آمده را گزارش می کنیم.

قسمت ۱

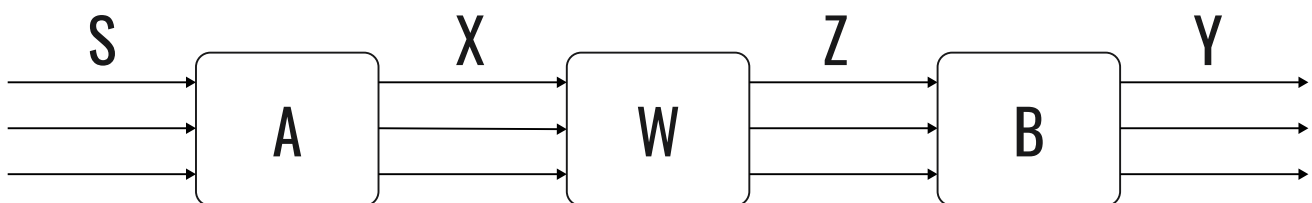
در این قسمت به دنبال پیاده سازی روش بیشینه سازی kurt با رویکرد fixed-point هستیم. به طور خلاصه، این روش به صورت زیر می باشد:

$$f(B) = \sum_{m=1}^M H(y_m) \rightarrow \begin{cases} b_i^{(k+1)} \leftarrow E \left\{ Z \left(b_i^{(k)T} Z \right)^3 \right\} - 3b_i^{(k)} \\ b \leftarrow \frac{b}{\|b\|_2} \\ b_i = \left(I - \begin{bmatrix} b_1^T \\ b_2^T \\ \vdots \\ b_{i-1}^T \end{bmatrix} \right) \end{cases}$$

Score Function: $|kurt(y)|$

$$kurt(y) = E\{y^4\} - 3(E\{y^2\})^2 \rightarrow \text{if } y \sim N(0, \sigma^2) \Rightarrow kurt(y) = 0$$

تصویر ۲.۱، نمودار کلی مسئله BSS در این الگوریتم را نشان می دهد.



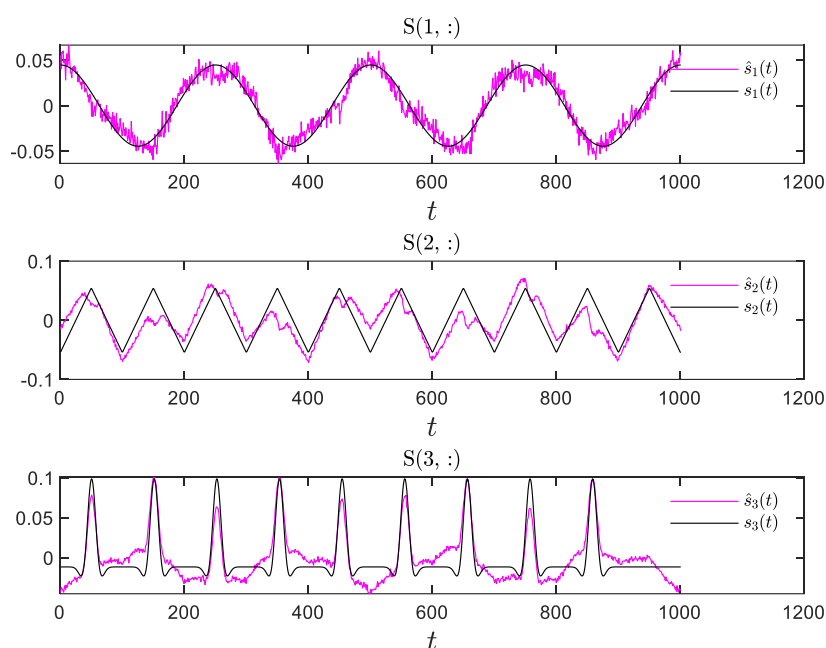
تصویر ۲.۱: نمودار کلی مسئله BSS برای fixed-point kurt

حال، ماتریس بدست آمده از ضرب ماتریس جدا کننده و ماتریس مخلوط کننده اصلی را بدست می آوریم. این ماتریس از رابطه $permutation_matrix = B \times W \times A$ بدست می آید. این ماتریس به صورت زیر می باشد:

$$permutation_matrix = \begin{bmatrix} -0.1437 & -1.1435 & 0.6589 \\ 0.6250 & -0.0132 & 0.7620 \\ -0.7552 & 0.1647 & 0.5369 \end{bmatrix}$$

همان طور که مشاهده می شود درایه های نشان داده شده باعث شده تا ماتریس بدست آمده از یک ماتریس permutation matrix دور باشد.

تصویر ۲.۲ مقایسه منابع تخمین زده شده از مشاهدات نویزی با منابع اصلی را نشان می دهد.



تصویر ۲.۲: مقایسه منابع تخمین زده شده از مشاهدات نویزی با منابع اصلی

قسمت ۲

رابطه خطا از فرمول زیر بدست می آید:

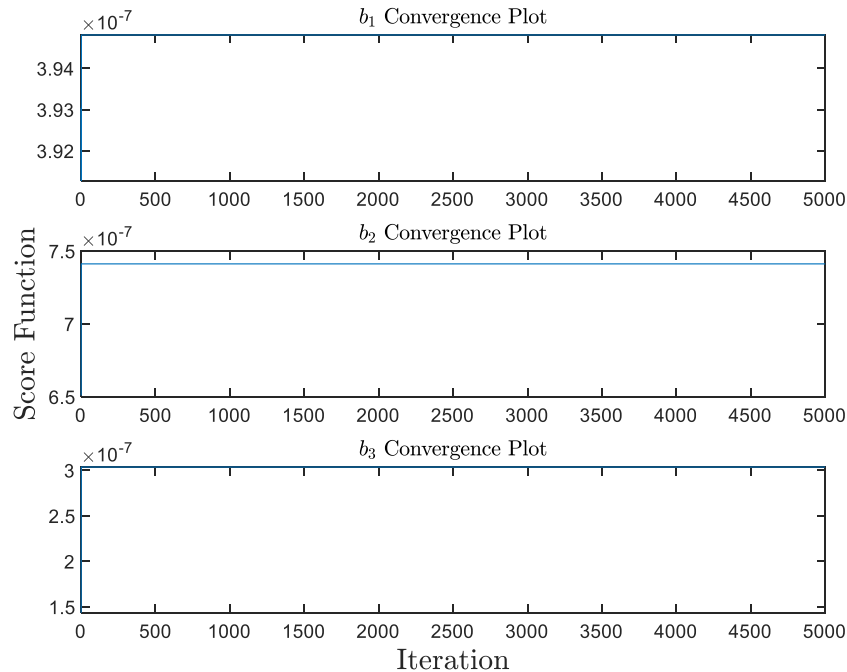
$$E = \frac{\|\hat{S} - S\|_F^2}{\|S\|_F^2}$$

نتایج بدست آمده از یک بار اجرای کد به صورت زیر می باشد:

$$E = 0.30682$$

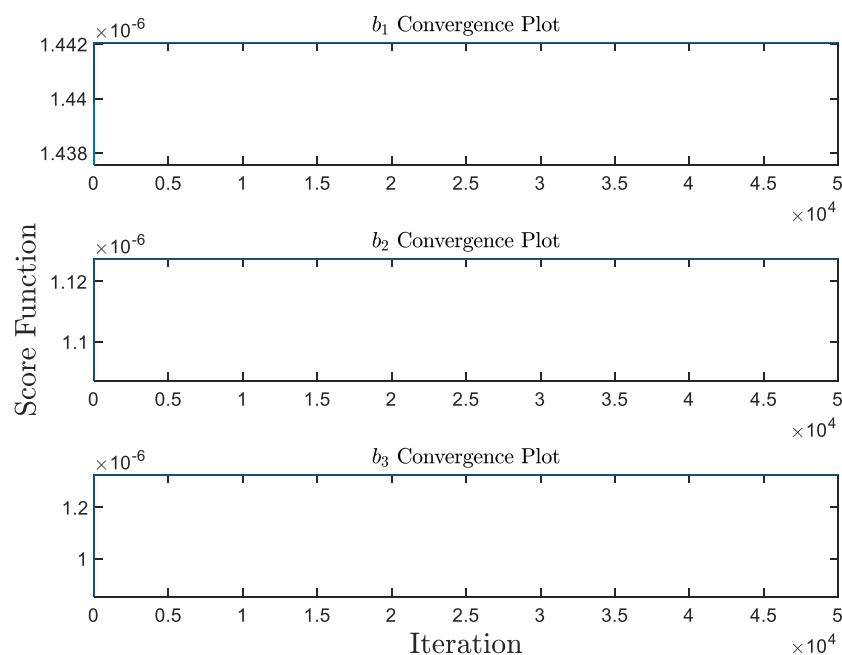
قسمت ۳

تابع هدف برابر با $f(B) = |kurt(y)|$ می باشد. نمودار آن را بر حسب شماره **iteration** رسم می کنیم. نمودار این تابع هدف در تصویر ۲.۳ آمده است.



تصویر ۲.۳: نمودار تابع هدف بر حسب شماره **iteration** (5000 iteration)

برای مشاهده بهتر این تابع هدف، تعداد **iteration** ها را بیشتر می کنیم (10 برابر می کنیم) و مجدداً این نمودار را رسم می کنیم. تصویر ۲.۴ این نمودار را نشان می دهد.



تصویر ۲.۴: نمودار تابع هدف بر حسب شماره **iteration** (50000 iteration)

همان طور که مشاهده می شود، تابع هدف مورد نظر طی **iteration** ها زیاد می شود و ثابت می ماند. به طور کلی سرعت همگرایی این روش بسیار زیاد می باشد.

بخش سوم

مقدمه

در این بخش به دنبال این هستیم که جداسازی منابع کور را با فرض استقلال منابع حل کنیم.

در قسمت ۱، به دنبال این هستیم که ماتریس جدا کننده را بدست آوریم. در ادامه و در قسمت ۲، ابهام منابع و ابهام ترتیب را حذف می کنیم و خطا را بدست می آوریم.

در قسمت ۳، نمودار همگرایی را رسم و نتایج بدست آمده را گزارش می کنیم.

قسمت ۱

در این قسمت به دنبال پیاده سازی روش بیشینه سازی kurt با رویکرد GP هستیم. به طور خلاصه، این روش به صورت زیر می باشد:

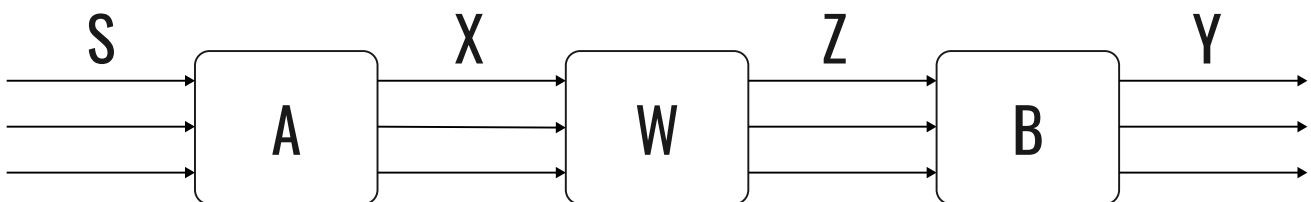
$$f(B) = \sum_{m=1}^M H(y_m) \rightarrow \begin{cases} b_i^{(k+1)} \leftarrow b_i^{(k)} - \mu \frac{\partial f}{\partial b_i} \\ b \leftarrow \frac{b}{\|b\|_2} \\ b_i = \left(I - \begin{bmatrix} b_1^T \\ b_2^T \\ \vdots \\ b_{i-1}^T \end{bmatrix} \right) \end{cases}$$

$$\text{Score Function} \propto [E\{G(y) - E\{G(v)\}}]^2 \Rightarrow b = \text{argmax}(b)$$

$$\frac{\partial f}{\partial b} = f_1(b) \times \frac{\partial f_1}{\partial b} = f_1(b)[E\{Zg(b^T Z)\}] \rightarrow f_1(b) = E\{G(y) - E\{G(v)\}}\}$$

$$\Rightarrow v \sim N(0, \sigma^2)$$

تصویر ۳.۱، نمودار کلی مسئله BSS در این الگوریتم را نشان می دهد.



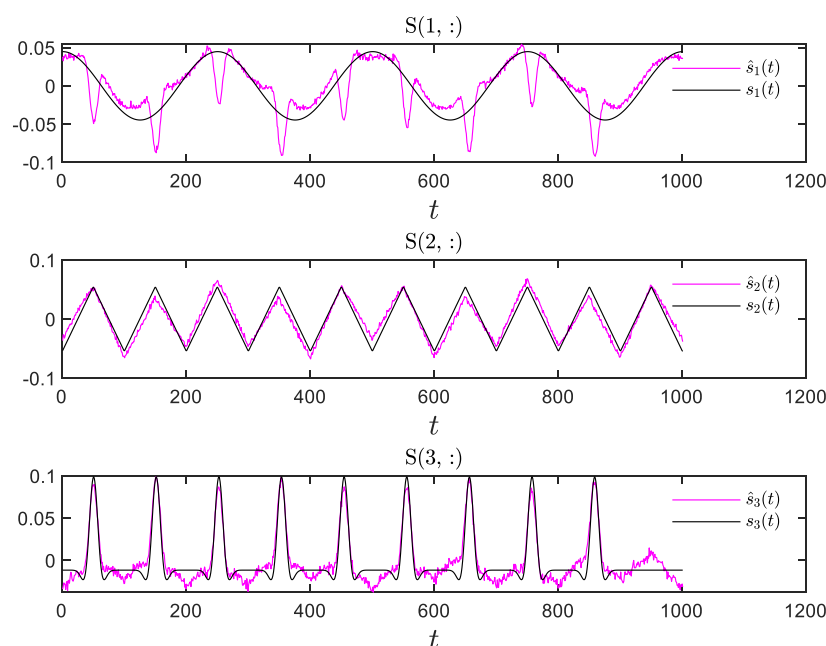
تصویر ۳.۱: نمودار کلی مسئله BSS برای kurt GP

حال، ماتریس بدست آمده از ضرب ماتریس جدا کننده و ماتریس مخلوط کننده اصلی را بدست می آوریم. این ماتریس از رابطه $permutation_matrix = B \times W \times A$ بدست می آید. این ماتریس به صورت زیر می باشد:

$$permutation_matrix = \begin{bmatrix} -0.0434 & 1.0687 & -0.9062 \\ -0.9873 & -0.1091 & 0.0149 \\ -0.0700 & 0.4252 & 0.6939 \end{bmatrix}$$

همان طور که مشاهده می شود درایه های نشان داده شده باعث شده تا ماتریس بدست آمده از یک ماتریس permutation matrix دور باشد.

تصویر ۳.۲ مقایسه منابع تخمین زده شده از مشاهدات نویزی با منابع اصلی را نشان می دهد.



تصویر ۳.۲: مقایسه منابع تخمین زده شده از مشاهدات نویزی با منابع اصلی

قسمت ۲

رابطه خطا از فرمول زیر بدست می آید:

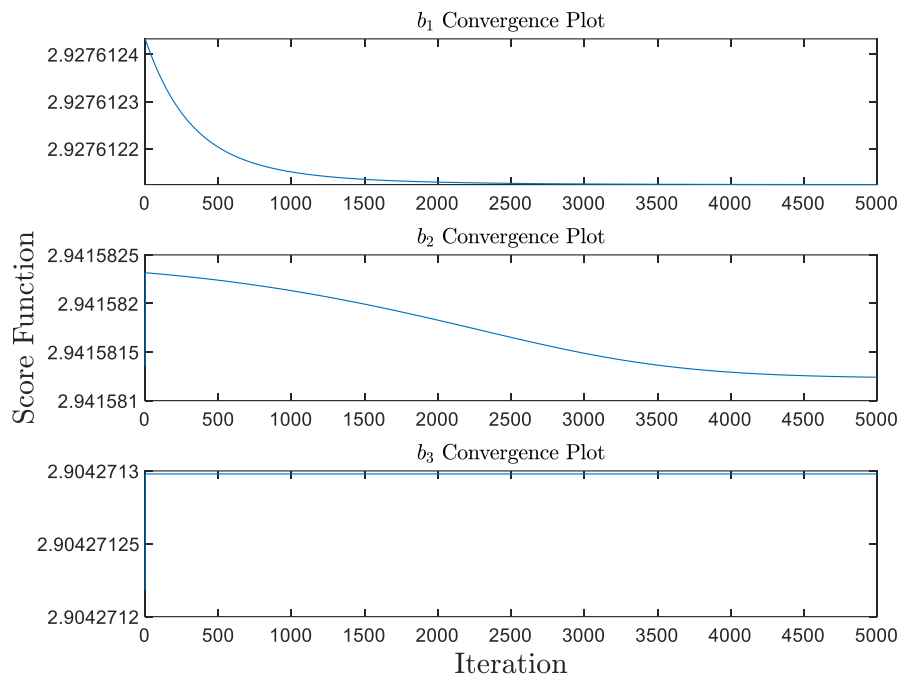
$$E = \frac{\|\hat{S} - S\|_F^2}{\|S\|_F^2}$$

نتایج بدست آمده از یک بار اجرای کد به صورت زیر می باشد:

$$E = 0.20242$$

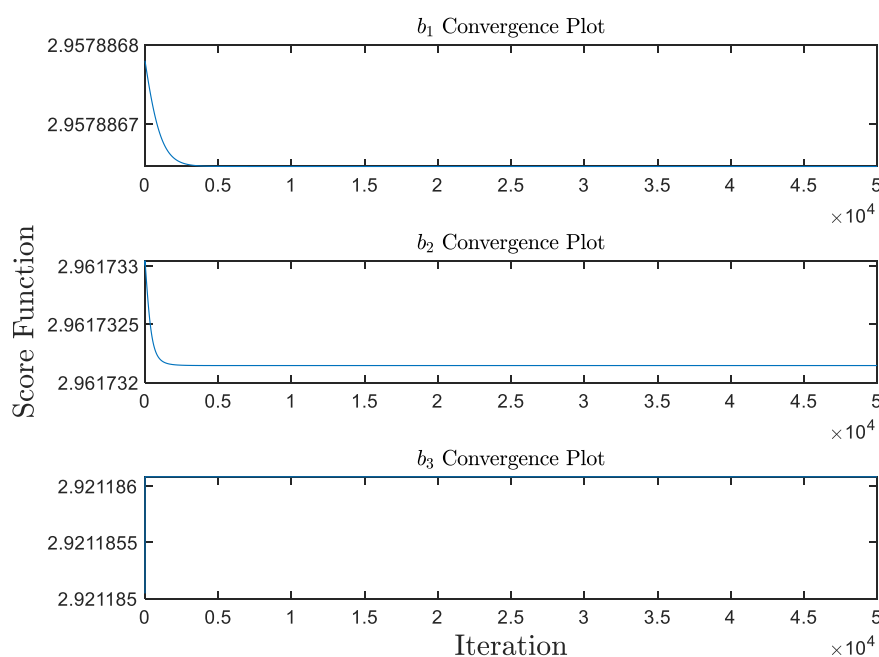
قسمت ۳

تابع هدف برابر با $f(B) = [E\{G(y) - E\{G(v)\}}]^2$ می باشد. نمودار آن را بر حسب شماره iteration رسم می کنیم. نمودار این تابع هدف در تصویر ۳.۳ آمده است.



تصویر ۳.۳: نمودار تابع هدف بر حسب شماره iteration (5000 iteration)

برای مشاهده بهتر این تابع هدف، تعداد iteration ها را بیشتر می کنیم (10 برابر می کنیم) و مجدداً این نمودار را رسم می کنیم. تصویر ۳.۴ این نمودار را نشان می دهد.



تصویر ۳.۴: نمودار تابع هدف بر حسب شماره iteration (50000 iteration)

همان طور که مشاهده می شود، تابع هدف مورد نظر طی **iteration** ها تغییر می کند و نزولی است. اما تابع سوم بر خلاف دو تابع بالا است. دلیل این اتفاق این است که تابع سوم به دلیل یکتا بودن به علت تعامل با دو تابع دیگر، در یک **iteration** به جواب نهایی رسیده است و تابع هدف آن از صفر (فرض شده) به مقداری ثابت تغییر یافته است.

بخش چهارم

مقدمه

در این بخش به دنبال این هستیم که جداسازی منابع کور را با فرض استقلال منابع حل کنیم.

در قسمت ۱، به دنبال این هستیم که ماتریس جدا کننده را بدست آوریم. در ادامه و در قسمت ۲، ابهام منابع و ابهام ترتیب را حذف می کنیم و خطا را بدست می آوریم.

در قسمت ۳، نمودار همگرایی را رسم و نتایج بدست آمده را گزارش می کنیم.

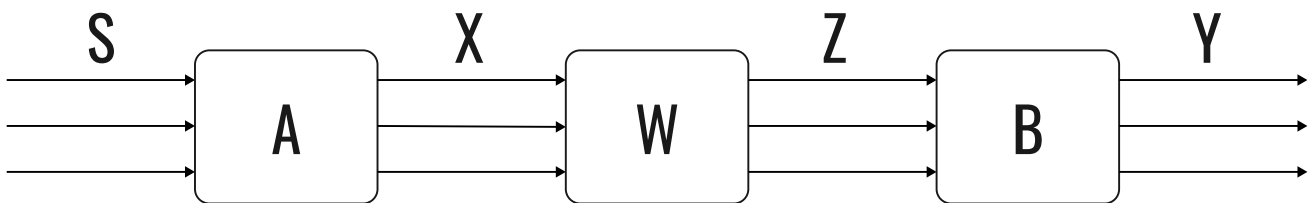
قسمت ۱

در این قسمت به دنبال پیاده سازی روش بیشینه سازی kurt با رویکرد fixed-point (اصلاح شده) هستیم. به طور خلاصه، این روش به صورت زیر می باشد:

$$f(B) = \sum_{m=1}^M H(y_m) \rightarrow \begin{cases} b_i^{(k+1)} \leftarrow E\{Zg(y)\} - E\{g'(y)b\} \\ b \leftarrow \frac{b}{\|b\|_2} \\ b_i = \left(I - \begin{bmatrix} b_1^T \\ b_2^T \\ \vdots \\ b_{i-1}^T \end{bmatrix} \right) \end{cases}$$

$$\text{Score Function} \propto [E\{G(y) - E\{G(v)\}}]^2 \Rightarrow b = \operatorname{argmax}(b)$$

تصویر ۴.۱، نمودار کلی مسئله BSS در این الگوریتم را نشان می دهد.



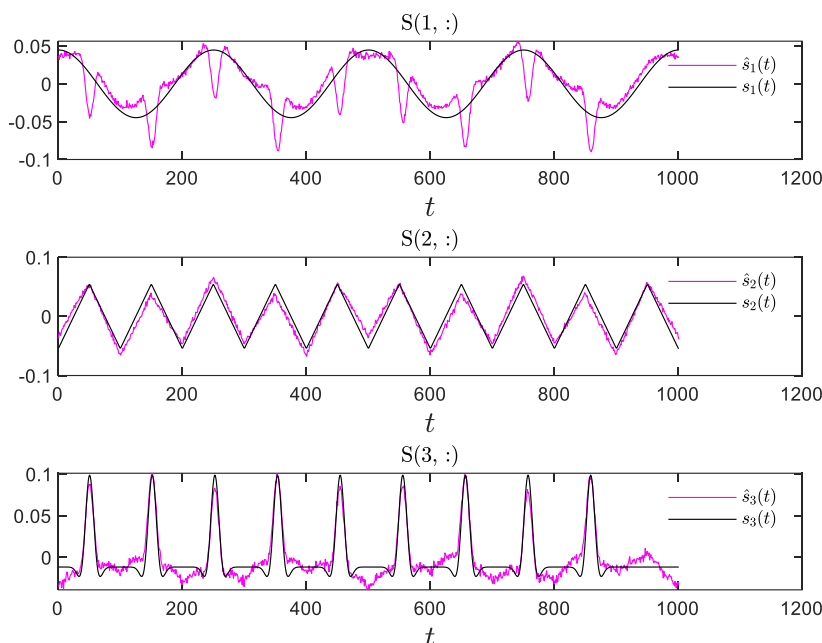
تصویر ۴.۱: نمودار کلی مسئله BSS برای kurt deflation (اصلاح شده)

حال، ماتریس بدست آمده از ضرب ماتریس جدا کننده و ماتریس مخلوط کننده اصلی را بدست می آوریم. این ماتریس از رابطه $\text{permutation_matrix} = B \times W \times A$ بدست می آید. این ماتریس به صورت زیر می باشد:

$$\text{permutation_matrix} = \begin{bmatrix} 0.9897 & 0.0699 & -0.0751 \\ -0.0421 & 1.0721 & -0.9009 \\ 0.0179 & 0.4248 & 0.6970 \end{bmatrix}$$

همان طور که مشاهده می شود درایه های نشان داده شده باعث شده تا ماتریس بدست آمده از یک ماتریس permutation matrix دور باشد.

تصویر ۴.۲ مقایسه منابع تخمین زده شده از مشاهدات نویزی با منابع اصلی را نشان می دهد.



تصویر ۴.۲: مقایسه منابع تخمین زده شده از مشاهدات نویزی با منابع اصلی

قسمت ۲

رابطه خطا از فرمول زیر بدست می آید:

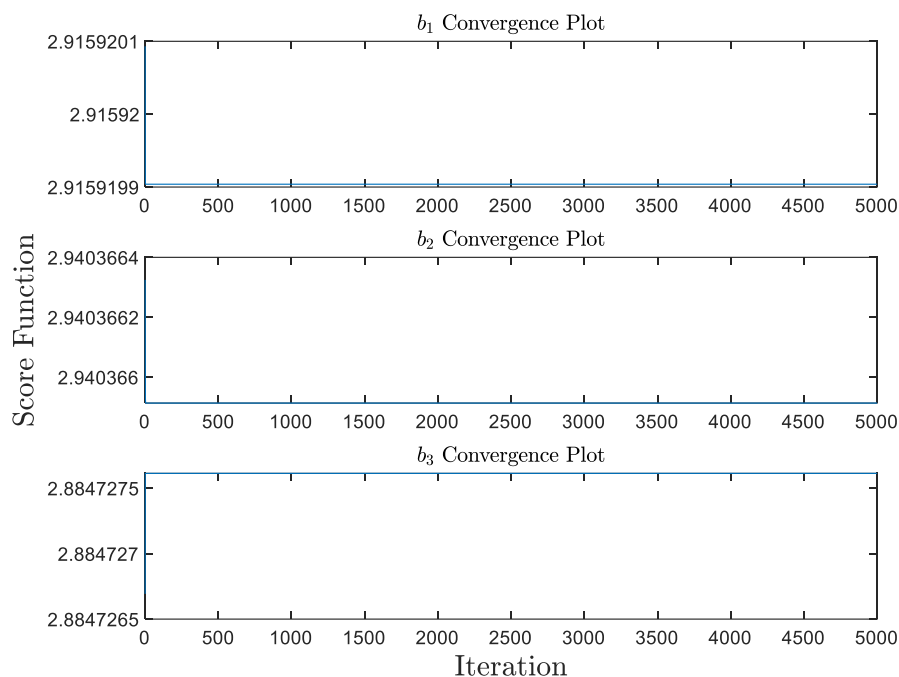
$$E = \frac{\|\hat{S} - S\|_F^2}{\|S\|_F^2}$$

نتایج بدست آمده از یک بار اجرای کد به صورت زیر می باشد:

$$E = 0.20242$$

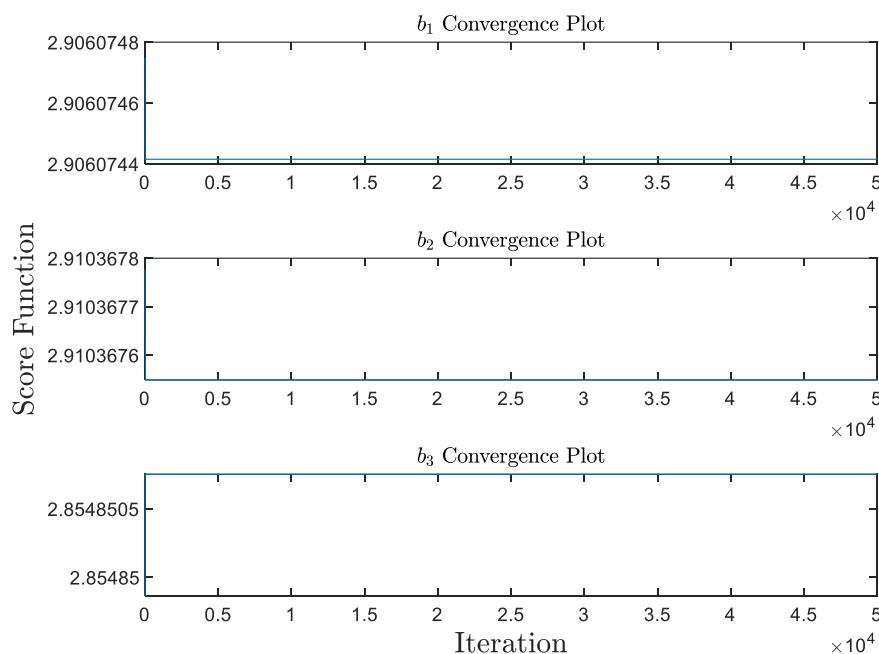
قسمت ۳

تابع هدف برابر با $f(B) = [E\{G(y) - E\{G(v)\}}]^2$ می باشد. نمودار آن را بر حسب شماره iteration رسم می کنیم. نمودار این تابع هدف در تصویر ۴.۳ آمده است.



تصویر ۴.۳: نمودار تابع هدف بر حسب شماره $iteration$ (5000 iteration)

برای مشاهده بهتر این تابع هدف، تعداد $iteration$ ها را بیشتر می کنیم (10 برابر می کنیم) و مجدداً این نمودار را رسم می کنیم. تصویر ۴.۴ این نمودار را نشان می دهد.



تصویر ۴.۴: نمودار تابع هدف بر حسب شماره $iteration$ (50000 iteration)

همان طور که مشاهده می شود، تابع هدف مورد نظر طی $iteration$ ها تغییر می کند و نزولی است. اما تابع سوم بر خلاف دو تابع بالا است. دلیل این اتفاق این است که تابع سوم به دلیل یکتا بودن به علت تعامد با دو تابع دیگر، در یک $iteration$ به جواب نهایی رسیده است و تابع هدف آن از صفر (فرض شده) به مقداری ثابت تغییر یافته است.

بخش پنجم

مقایسه الگوریتم‌ها

نکته: برای این که کد های ارسالی الگوریتم‌ها را بیان می کنند، هر تغییراتی که برای این قسمت در کد را انجام داده ام به حالت قبل باز می گردانم تا کد ها طولانی نشوند و افزون بر آن، هر بار یک ماتریس تصادفی متفاوت به عنوان ماتریس B تعریف شود.

برای مقایسه الگوریتم‌ها باید یک ماتریس تصادفی B به عنوان `initial matrix` بدهیم. در نتیجه ماتریس B را برابر با دیتایی که در `B_init.mat` وجود دارد تعریف می کنیم.

اگر شرایط را اعمال کنیم و از دستور `tic-toc` برای محاسبه زمان اجرای الگوریتم استفاده کنیم، به نتایج زیر می رسم:

اجرا با $iterNum = 5000$

• الگوریتم بخش اول

○ زمان اجرای کد: $1.364805 [s]$

○ خطای تعریف شده: $E = 0.24635$

○ بیشترین تعداد `iteration` برای همگرایی یک سطر: تقریباً 5000 iteration

• الگوریتم بخش دوم

○ زمان اجرای کد: $0.824397 [s]$

○ خطای تعریف شده: $E = 0.49648$

○ بیشترین تعداد `iteration` برای همگرایی یک سطر: بسیار اندک

• الگوریتم بخش سوم

○ زمان اجرای کد: $0.700543 [s]$

○ خطای تعریف شده: $E = 0.22055$

○ بیشترین تعداد `iteration` برای همگرایی یک سطر: تقریباً 3000 iteration

• الگوریتم بخش چهارم

○ زمان اجرای کد: $1.255933 [s]$

○ خطای تعریف شده: $E = 0.20242$

○ بیشترین تعداد `iteration` برای همگرایی یک سطر: بسیار اندک

اجرا با $iterNum = 50000$

• الگوریتم بخش اول

○ زمان اجرای کد: $13.699159 [s]$ ○ خطای تعریف شده: $E = 0.20243$ ○ بیشترین تعداد $iteration$ برای همگرایی یک سطر: تقریباً $5000 iteration$

• الگوریتم بخش دوم

○ زمان اجرای کد: $8.677623 [s]$ ○ خطای تعریف شده: $E = 0.49648$ ○ بیشترین تعداد $iteration$ برای همگرایی یک سطر: بسیار اندک

• الگوریتم بخش سوم

○ زمان اجرای کد: $6.883684 [s]$ ○ خطای تعریف شده: $E = 0.20242$ ○ بیشترین تعداد $iteration$ برای همگرایی یک سطر: تقریباً $2000 iteration$

• الگوریتم بخش چهارم

○ زمان اجرای کد: $12.379115 [s]$ ○ خطای تعریف شده: $E = 0.20242$ ○ بیشترین تعداد $iteration$ برای همگرایی یک سطر: بسیار اندک**بررسی و مقایسه نهایی**

با بررسی داده‌های بالا متوجه می‌شویم که الگوریتم **fastICA** نهایی با آن که در تعداد $iteration$ های مشخص و یکسان تقریباً بدترین الگوریتم از نظر زمان است، اما زمان همگرایی آن بسیار کم می‌باشد؛ به طوری که شیب نمودار تابع هدف آن بسیار سریع است. از نظر سرعت همگرایی دو الگوریتمی که با رویکرد **fixed-point** هستند همگرایی سریع‌تری دارند.

همچنین، اگر از نظر معیار E الگوریتم‌ها را بررسی کنیم، متوجه می‌شویم که الگوریتم بخش دوم به طرز قابل توجهی بدتر از سایر الگوریتم‌ها می‌باشد.

به طور کلی می توان گفت:

convergence time:

$$\text{fastICA}(\text{fixed} - \text{point}) \approx \text{kurt}(\text{fixed} - \text{point}) < \text{fastICA}(G.P) < \text{kurt}(\text{deflation})$$

Error (E):

$$\text{fastICA}(\text{fixed} - \text{point}) \approx \text{fastICA}(G.P) \leq \text{kurt}(\text{deflation}) < \text{fastICA}(G.P)$$